

เอกสารประกอบใบงานปฏิบัติการที่ 3 วิชา ENG23 3054 OPERATING SYSTEMS

สาขาวิชาวิศวกรรมคอมพิวเตอร์ สำนักวิชาวิศวกรรมศาสตร์

มหาวิทยาลัยเทคโนโลยีสุรนารี

Shell script เป็นไฟล์ที่เขียนด้วยภาษาที่สามารถสร้างด้วย editor ชนิดไหนก็ได้ เช่น vi, nano, gedit, emacs เป็นต้น

1. เริ่มการใช้งาน สร้างไฟล์ชื่อ shell_script และเข้าไปใน folder

1.1 เปิด terminal และใช้คำสั่ง nano test1.sh (test1.sh เป็นชื่อ script) เพื่อเรียกโปรแกรม text editor

1.2 พิมพ์โค้ดลงไป แล้ว save โดยกด ctrl+o ออกโดยกด ctrl+x

```
#!/bin/sh
echo "Hello world"
```

1.3 ก่อนจะรัน script ต้องเปลี่ยนโหมดการทำงานของไฟล์นี้ให้สามารถ exec ได้เสียก่อน โดยใช้คำสั่ง chmod ให้เรา (user) สามารถรันได้

1.4 ใช้คำสั่งรัน ./test1.sh

ผลที่ได้

```
os@os-VirtualBox:~/shell_script$ ./test1.sh
Hello world
```

ตัวแปร

```
#!/bin/sh
var1='Suranaree University of Technology'
var2='CPE'
echo 'I am studying in ' $var2 ' at ' $var1
```

ผลที่ได้

```
I am studying in CPE at Suranaree University of Technology
```

- โปรแกรมบวกเลข

```
#!/bin/sh
num1=5
num2=10
num_add=$((num2+$num1))
num_minus=$((num2-$num1))
num_multiple=$((num2*$num1))
num_divide=$((num2/$num1))
echo $num_add $num_minus $num_multiple $num_divide
```

ผลที่ได้

```
15 5 50 2
```

- โปรแกรมแสดงชื่อ user ที่ login วันเวลาปัจจุบัน

```
#!/bin/sh
echo "Hello $USER"
echo " Today is "`date +%d-%B-%y`"
echo " Today is "`date +%A-%d-%m-%y`"
echo "Time is "`date +%k:%M:%S`"
```

ผลที่ได้

```
Hello os
Today is 17-September-13
Today is Tuesday-17-09-13
Time is 15:27:13
```

<http://www.adminschoice.com/unix-date-format-examples/>

if else condition

```
#!/bin/sh
num1=10
num2=1
num3=1

if [ $num1 -gt $num2 ]
then
    echo $num1 'is higher than' $num2
else
    echo $num1 'is lower than' $num2
fi

if [ $num1 = $num2 ]
then
    echo $num1 ' equal' $num2
elif [ $num2 = $num3 ]
then
    echo $num2 ' equal' $num3
elif [ $num1 = $num3 ]
then
```

```
        echo $num1 'equal' $num3
else
        echo 'These number are not equal'
fi
```

ผลที่ได้

```
10 is higher than 1
1 equal 1
```

คำสั่งเกี่ยวกับจำนวนเต็ม

int1 -eq int2 เป็นจริงเมื่อ int1 เท่ากับ int2

int1 -ne int2 เป็นจริงเมื่อ int1 ไม่เท่ากับ int2

int1 -gt int2 เป็นจริงเมื่อ int1 มากกว่า int2

int1 -ge int2 เป็นจริงเมื่อ int1 มากกว่า หรือ เท่ากับ int2

int1 -le int2 เป็นจริงเมื่อ int1 น้อยกว่า int2 หรือ เท่ากับ int2

int1 -lt int2 เป็นจริงเมื่อ int1 น้อยกว่า

คำสั่งเกี่ยวกับ string

str1 = str2 เป็นจริงเมื่อ str1 เหมือนกับ str2

str1 != str2 เป็นจริงเมื่อ str1 ไม่เหมือนกับ str2

Str เป็นจริงเมื่อ str ไม่เป็น ค่าว่าง

-n str เป็นจริงเมื่อ str มีความยาวมากกว่า 0

-z str เป็นจริงเมื่อ str มีความยาวเป็น 0

- โปรแกรมถามอายุ โดยรับค่าจากผู้ใช้

```
#!/bin/sh
echo "Please enter your age:"
read AGE

if [ "$AGE" -lt 20 ] || [ "$AGE" -ge 50 ]; then
    echo "Sorry, you are out of the age range."
elif [ "$AGE" -ge 20 ] && [ "$AGE" -lt 30 ]; then
    echo "You are in your 20s"
elif [ "$AGE" -ge 30 ] && [ "$AGE" -lt 40 ]; then
    echo "You are in your 30s"
elif [ "$AGE" -ge 40 ] && [ "$AGE" -lt 50 ]; then
```

ผลที่ได้

```
echo "You are in your 40s"  
fi
```

```
Please enter your age:  
25  
You are in your 20s
```

While loop

ผลที่ได้

```
#!/bin/sh  
num=1  
while [ $num -le 10 ]  
do  
    echo 'Round' $num  
    num=$((num+1))  
done
```

```
Round 1  
Round 2  
Round 3  
Round 4  
Round 5  
Round 6  
Round 7  
Round 8  
Round 9  
Round 10
```

For loop

ผลที่ได้

```
#!/bin/sh  
for i in 1 2 3  
do  
    echo 'SUT' $i  
done
```

```
SUT 1
```

```
SUT 2
SUT 3
```

For loop อีกรูปแบบหนึ่ง

```
#!/bin/bash
#{START..END..INCREMENT}
for i in {0..10..2}
do
    echo "Welcome $i times"
done
```

ผลที่ได้

```
Welcome 0 times
Welcome 2 times
Welcome 4 times
Welcome 6 times
Welcome 8 times
Welcome 10 times
```

Shell script สามารถใช้งานร่วมกับคำสั่งต่างๆของ linux ได้ เช่น

```
#!/bin/sh
ls -al
```

จากนั้นก็รันไฟล์นี้ (./filename.sh)จะได้ list ของไฟล์ใน directory ปัจจุบัน

While do VS until do

- While จะออกจะ loop ถ้า ผิดเงื่อนไข
- Until จะออกจะ loop ถ้าเข้าเงื่อนไข

```
x=angel
name=dangel
until [ "$name" = "$x" ]
do
    echo "Your guess: c"
    read name
done
echo Right Guess!!!
```

การเขียนลงไฟล์

สร้างไฟล์ใหม่ (Create)

```
#!/bin/sh
echo "Hello world" > newfile.txt
```

เขียนต่อไฟล์เดิม (append)

```
#!/bin/sh
echo "Hello world" >> newfile.txt
```

ตัวอย่าง

ต้องการแสดง CPU and I/O statistics

```
#!/bin/sh
echo "===== " > iostatDisplay.txt
iostat >> iostatDisplay.txt
```

เปิด iostatDisplay.txt

```
=====
Linux 3.5.0-37-generic (os-VirtualBox) 09/17/2013 _i686_ (1 CPU)

avg-cpu:  %user  %nice %system %iowait  %steal   %idle
           0.89   0.95  26.30   0.91   0.00  70.95

Device:            tps    kB_read/s    kB_wrtn/s    kB_read    kB_wrtn
scd0                0.00         0.01         0.00         70         0
sda                 11.17        74.41        103.14      1017679      1410596
```

Crontab

Crontab คือคำสั่งที่จะทำงานตามเวลาที่กำหนด ซึ่งจะช่วยให้เราสะดวกขึ้นมากในการที่จะต้องสั่งงานบางอย่างซ้ำๆ กันหลายๆ ครั้ง อาจจะเป็นทุกๆ ชั่วโมง ทุกๆ วัน หรือ ทุกๆ เดือน เช่นการสั่งให้ Server ทำ Backup ทุกๆ สัปดาห์

คำสั่งและ option ของ crontab มีดังนี้

crontab filename การนำเอาคำสั่ง crontab เข้ามาจาก ไฟล์อื่น

crontab -e แก้ไข crontab ปัจจุบัน (ส่วนมากเราจะเพิ่มคำสั่งลงไปนี่เลย)

crontab -l ดูคำสั่ง crontab ทั้งหมดที่มีอยู่

crontab -r ลบคำสั่ง crontab ที่มีทั้งหมด

crontab -u user เป็นคำสั่งของผู้ดูแลระบบเท่านั้น เพื่อใช้ดู แก้ไข ลบ crontab ของ user แต่ละคน

รูปแบบของคำสั่ง crontab มีทั้งหมด 6 fields ดังนี้

1 = minute มีค่า 0 - 59 เวลาเป็นนาที จะสั่งให้คำสั่งที่กำหนดทำงานทันทีเมื่อถึงนาทีที่กำหนด

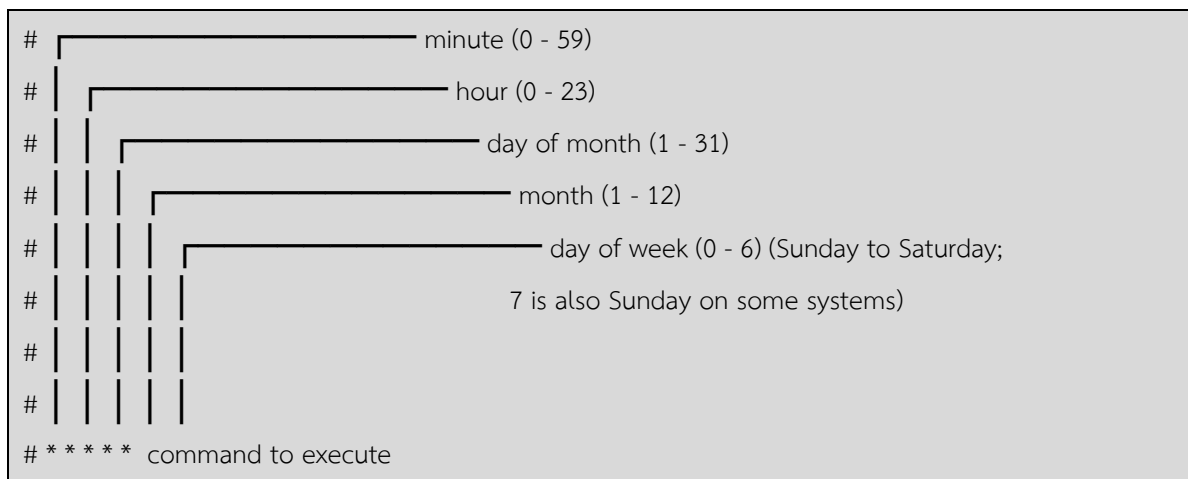
2 = hour มีค่า 0 - 23 เวลาเป็นชั่วโมง จะสั่งให้คำสั่งที่กำหนดทำงานทันทีเมื่อถึงชั่วโมงที่กำหนด

3 = day มีค่า 1 - 31 เวลาเป็นวัน จะสั่งให้คำสั่งที่กำหนดทำงานทันทีเมื่อถึงวันที่กำหนด

4 = month มีค่า 1 - 12 เวลาเป็นเดือน จะสั่งให้คำสั่งที่กำหนดทำงานทันทีเมื่อถึงเดือนที่กำหนด

5 = weekday มีค่า 0 - 6 วันของแต่ละสัปดาห์ มีค่าดังนี้ (อาทิตย์ = 0, จันทร์ = 1, อังคาร = 2, พุธ = 3, พฤหัส = 4, ศุกร์ = 5 , เสาร์ = 6)

6 = command คำสั่ง เราสามารถกำหนดคำสั่งได้มากมาย รวมทั้ง script ต่าง ๆ ตามที่เราต้องการ



ตัวอย่าง

การสั่งให้ /path/to/command ทำงานเมื่อเวลาผ่านไป 5 นาที หลังเที่ยงคืน ในทุกๆ วัน

```
5 0 * * * /path/to/command
```

การสั่งให้ /path/to/script.sh ทำงานเวลา 14.25น. ของวันที่ 1 ในทุกๆ เดือน

```
15 14 1 * * /path/to/script.sh
```

การสั่งให้ /path/to/script.sh ทำงานในทุกๆ 3 ชั่วโมง

```
0 */3 * * * /path/to/script.sh
```